

实验二 DBMS软件安装及数据定义、修改、导入实验报告

2023级图灵实验班 张昕跃 2023202300

1. 实验环境（计算机配置，操作系统，编程语言，编程工具等）

- 计算机配置: MacBook Air, Apple M3
- 操作系统: macOS Sequoia 15.1 (Build 24B83)
- 数据库管理系统 (DBMS): PostgreSQL 16.1
- 安装与管理工具: Homebrew, Iterm
- 数据库客户端: PostgreSQL
- 实验数据: TPC-H 数据集 (Scale Factor = 0.01)

2. 实验结果及分析

2.1 DBMS 安装过程

本次实验通过 macOS 的包管理器 Homebrew 安装 PostgreSQL。主要步骤包括更新 Homebrew、执行安装命令以及启动数据库服务。

因为装完了才发现要三张截图emmm，只能放安装好的结果了。

- 截图1: 支持的PostgreSql版本。

```

~ /De/d/tbl ➤ brew search postgresql
==> Formulae
postgresql-hll      postgresql@13      postgresql@16 ✓    postgres
postgresql@11       postgresql@14 ✓    postgresql@17
postgresql@12       postgresql@15      qt-postgresql

==> Casks
navicat-for-postgresql      posture-pal

If you meant "postgresql" specifically:
postgresql breaks existing databases on upgrade without human intervention.

See a more specific version to install with:
brew formulae | grep postgresql@
```

- 截图2: 执行 `brew install postgresql` 命令的终端界面。

```
~/De/d/tbl ➤ brew install postgresql@16
==> Auto-updating Homebrew...
Adjust how often this is run with `$_HOMEBREW_AUTO_UPDATE_SECS` or disable with
`$_HOMEBREW_NO_AUTO_UPDATE=1`. Hide these hints with `$_HOMEBREW_NO_ENV_HINTS=1` (
see `man brew`).
==> Auto-updated Homebrew!
Updated 2 taps (homebrew/core and homebrew/cask).
==> New Formulae
atomic_queue: C++14 lock-free queues
zuban: Python language server and type checker, written in Rust
==> New Casks
cmd: AI assistant for development in Xcode
coderabbit: AI code review CLI
folder-preview-pro: Quick Look extension for folders
rclone-ui: GUI for Rclone

You have 65 outdated formulae installed.

Warning: postgresql@16 16.10 is already installed and up-to-date.
To reinstall 16.10, run:
  brew reinstall postgresql@16
```

- 截图3：使用 `psql postgres` 成功连接到默认数据库的界面。

```
~/De/d/tbl ➤ psql postgres
psql (16.10 (Homebrew))
输入 "help" 来获取帮助信息。

postgres=#
```

2.2 执行DDL、DML语句

成功连接到新建的 `tpch_db` 数据库后，首先执行了数据定义语言（DDL）语句，创建了 TPC-H 标准的 8 张数据表。

随后，执行了数据操纵语言（DML）中的 `ALTER TABLE` 语句，为各表添加了主键和外键约束，以保证数据的实体完整性和引用完整性。

- 截图4：执行 `CREATE TABLE` 语句，成功创建 8 张表的终端界面。用 `\dt` 命令的输出来验证。

```
psql tpch_db
L_QUANTITY    DECIMAL(15,2) NOT NULL,
L_EXTENDEDPRICE DECIMAL(15,2) NOT NULL,
L_DISCOUNT    DECIMAL(15,2) NOT NULL,
L_TAX          DECIMAL(15,2) NOT NULL,
L_RETURNFLAG   CHAR(1) NOT NULL,
L_LINESTATUS   CHAR(1) NOT NULL,
L_SHIPDATE     DATE NOT NULL,
L_COMMITDATE   DATE NOT NULL,
L_RECEIPTDATE  DATE NOT NULL,
L_SHIPINSTRUCT CHAR(25) NOT NULL,
L_SHIPMODE     CHAR(10) NOT NULL,
L_COMMENT      VARCHAR(44) NOT NULL);
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
tpch_db=# \dt
          关联列表
架构模式 | 名称 | 类型 | 拥有者
-----+-----+-----+-----
public   | customer | 数据表 | 0ximio
public   | lineitem | 数据表 | 0ximio
public   | nation   | 数据表 | 0ximio
public   | orders   | 数据表 | 0ximio
public   | part     | 数据表 | 0ximio
public   | partsupp | 数据表 | 0ximio
public   | region   | 数据表 | 0ximio
public   | supplier | 数据表 | 0ximio
(8 行记录)
tpch_db=#
```

- 截图5：执行 `ALTER TABLE` 语句，为各表添加主键和外键约束，终端显示 `ALTER TABLE` 成功信息。

```
架构模式 | 名称 | 类型 | 拥有者
-----+-----+-----+-----
public | customer | 数据表 | 0ximio
public | lineitem | 数据表 | 0ximio
public | nation | 数据表 | 0ximio
public | orders | 数据表 | 0ximio
public | part | 数据表 | 0ximio
public | partsupp | 数据表 | 0ximio
public | region | 数据表 | 0ximio
public | supplier | 数据表 | 0ximio
(8 行记录)

tpch_db=# ALTER TABLE REGION ADD PRIMARY KEY (R_REGIONKEY);
ALTER TABLE
tpch_db=# ALTER TABLE NATION ADD PRIMARY KEY (N_NATIONKEY);
ALTER TABLE
tpch_db=# ALTER TABLE NATION ADD FOREIGN KEY (N_REGIONKEY) REFERENCES REGION(R_REGIONKEY);
ALTER TABLE
tpch_db=# \dt
          关联列表
架构模式 | 名称 | 类型 | 拥有者
-----+-----+-----+-----
public | customer | 数据表 | 0ximio
public | lineitem | 数据表 | 0ximio
public | nation | 数据表 | 0ximio
public | orders | 数据表 | 0ximio
public | part | 数据表 | 0ximio
public | partsupp | 数据表 | 0ximio
public | region | 数据表 | 0ximio
public | supplier | 数据表 | 0ximio
(8 行记录)

tpch_db=# ALTER TABLE PART ADD PRIMARY KEY (P_PARTKEY);
ALTER TABLE
tpch_db=# ALTER TABLE SUPPLIER ADD PRIMARY KEY (S_SUPPKEY);
ALTER TABLE
tpch_db=# ALTER TABLE SUPPLIER ADD FOREIGN KEY (S_NATIONKEY) REFERENCES NATION(N_NATIONKEY);
ALTER TABLE
tpch_db=# ALTER TABLE PARTSUPP ADD PRIMARY KEY (PS_PARTKEY, PS_SUPPKEY);
ALTER TABLE
tpch_db=# ALTER TABLE PARTSUPP ADD FOREIGN KEY (PS_PARTKEY) REFERENCES PART(P_PARTKEY);
ALTER TABLE
tpch_db=# ALTER TABLE PARTSUPP ADD FOREIGN KEY (PS_SUPPKEY) REFERENCES SUPPLIER(S_SUPPKEY);
ALTER TABLE
tpch_db=# ALTER TABLE CUSTOMER ADD PRIMARY KEY (C_CUSTKEY);
ALTER TABLE
tpch_db=# ALTER TABLE CUSTOMER ADD FOREIGN KEY (C_NATIONKEY) REFERENCES NATION(N_NATIONKEY);
ALTER TABLE
tpch_db=# ALTER TABLE ORDERS ADD PRIMARY KEY (O_ORDERKEY);
ALTER TABLE
tpch_db=# ALTER TABLE ORDERS ADD FOREIGN KEY (O_CUSTKEY) REFERENCES CUSTOMER(C_CUSTKEY);
ALTER TABLE
```

2.3 数据生成与导入

使用 PostgreSQL 提供的 `COPY` 命令从预先生成的 `.tbl` 数据文件中批量导入数据。该命令效率高，能够快速将大规模数据加载到数据库中。通过指定 `|` 为分隔符，成功将所有数据文件导入到对应的表中。

- 截图6：在 `psql` 中执行 `\copy` 命令导入数据的过程。截图显示了表的导入命令及其返回的成功信息（如 `COPY 1500`）。

```

tpch_db=# COPY part
FROM PROGRAM 'sed "s/|$/|" /Users/0ximio/Desktop/db_lab2/tbl/part.tbl'
WITH (FORMAT csv, DELIMITER '|', NULL '');
COPY 2000
tpch_db=# COPY region
FROM PROGRAM 'sed "s/|$/|" /Users/0ximio/Desktop/db_lab2/tbl/region.tbl'
WITH (FORMAT csv, DELIMITER '|', NULL '');
COPY 5
tpch_db=# COPY nation
FROM PROGRAM 'sed "s/|$/|" /Users/0ximio/Desktop/db_lab2/tbl/nation.tbl'
WITH (FORMAT csv, DELIMITER '|', NULL '');
COPY 25
tpch_db=# COPY customer
FROM PROGRAM 'sed "s/|$/|" /Users/0ximio/Desktop/db_lab2/tbl/customer.tbl'
WITH (FORMAT csv, DELIMITER '|', NULL '');
COPY 1500
tpch_db=# COPY supplier
FROM PROGRAM 'sed "s/|$/|" /Users/0ximio/Desktop/db_lab2/tbl/supplier.tbl'
WITH (FORMAT csv, DELIMITER '|', NULL '');
COPY 100
tpch_db=# COPY partsupp
FROM PROGRAM 'sed "s/|$/|" /Users/0ximio/Desktop/db_lab2/tbl/partsupp.tbl'
WITH (FORMAT csv, DELIMITER '|', NULL '');
COPY 8000
tpch_db=# COPY lineitem
FROM PROGRAM 'sed "s/|$/|" /Users/0ximio/Desktop/db_lab2/tbl/lineitem.tbl'
WITH (FORMAT csv, DELIMITER '|', NULL '');
COPY 60175
tpch_db=# COPY orders
FROM PROGRAM 'sed "s/|$/|" /Users/0ximio/Desktop/db_lab2/tbl/orders.tbl'
WITH (FORMAT csv, DELIMITER '|', NULL '');
COPY 15000
tpch_db=# ALTER TABLE LINEITEM ADD PRIMARY KEY (L_ORDERKEY, L_LINENUMBER);
ALTER TABLE
tpch_db=# ALTER TABLE LINEITEM ADD FOREIGN KEY (L_ORDERKEY) REFERENCES ORDERS(O_ORDERKEY);
ALTER TABLE
tpch_db=# ALTER TABLE LINEITEM ADD FOREIGN KEY (L_PARTKEY, L_SUPPKEY) REFERENCES PARTSUPP(PS_PARTKEY, PS_SUPPKEY);
ALTER TABLE
tpch_db=# █

```

2.4 数据查询与验证

数据导入完成后，进行了数据验证。首先，查询了各表的行数，结果与 TPC-H 在 SF=0.01 规模下的标准行数基本相符，证明数据导入完整。随后，执行了简单的单表查询和多表连接（JOIN）查询，均返回了预期的正确结果，表明数据库已搭建成功并可正常使用。

- 截图7：执行查询各表行数的 SQL 语句，返回的行数量都与预期一致（对应的customer 约 1,500 行，orders 约 15,000 行，lineitem 约 60,000 行）

```

SELECT 'region' AS table_name, count(*) AS row_count FROM region UNION ALL
SELECT 'nation', count(*) FROM nation UNION ALL
SELECT 'part', count(*) FROM part UNION ALL
SELECT 'supplier', count(*) FROM supplier UNION ALL
SELECT 'partsupp', count(*) FROM partsupp UNION ALL
SELECT 'customer', count(*) FROM customer UNION ALL
SELECT 'orders', count(*) FROM orders UNION ALL
SELECT 'lineitem', count(*) FROM lineitem;

```

```

tpch_db=# SELECT 'region' AS table_name, count(*) AS row_count FROM region UNION ALL
SELECT 'nation', count(*) FROM nation UNION ALL
SELECT 'part', count(*) FROM part UNION ALL
SELECT 'supplier', count(*) FROM supplier UNION ALL
SELECT 'partsupp', count(*) FROM partsupp UNION ALL
SELECT 'customer', count(*) FROM customer UNION ALL
SELECT 'orders', count(*) FROM orders UNION ALL
SELECT 'lineitem', count(*) FROM lineitem;
 table_name | row_count
-----+-----
 region    |         5
 nation    |        25
 part      |       2000
 supplier  |        100
 partsupp  |       8000
 customer  |       1500
 orders    |      15000
 lineitem  |      60175
(8 行记录)

tpch_db=#

```

- 截图8：执行简单的 SQL 查询语句。

```

SELECT * FROM nation LIMIT 5;

SELECT * FROM nation WHERE n_name = 'UNITED STATES';

```

```

tpch_db=# SELECT * FROM nation LIMIT 5;
 n_nationkey | n_name | n_regionkey | n_comment
-----+-----+-----+-----
 0 | ALGERIA | 0 | haggle. carefully final deposits detect slyly agai
 1 | ARGENTINA | 1 | al foxes promise slyly according to the regular accounts. bold requests alon
 2 | BRAZIL | 1 | y alongside of the pending deposits. carefully special packages are about the ironic forges. slyly special
 3 | CANADA | 1 | eas hang ironic, silent packages. slyly regular packages are furiously over the tithes. fluffily bold
 4 | EGYPT | 4 | y above the carefully unusual theodolites. final dugouts are quickly across the furiously regular d
(5 行记录)

tpch_db=# SELECT * FROM nation WHERE n_name = 'UNITED STATES';
 n_nationkey | n_name | n_regionkey | n_comment
-----+-----+-----+-----
 24 | UNITED STATES | 1 | y final packages. slow foxes cajole quickly. quickly silent platelets breach ironic accounts. unusual pinto be
(1 行记录)

```

- 截图9、10：执行带 JOIN 的复杂查询语句。

```

-- 查找前 5 位来自亚洲的客户
-- 通过 c_nationkey 将 customer 表与 nation 表关联，找到每个客户所在的国家
-- 再通过 n_regionkey 将 nation 表与 region 表关联，找到每个国家所属的地区
SELECT c_name, c_phone, n_name, r_name
FROM customer
JOIN nation ON c_nationkey = n_nationkey
JOIN region ON n_regionkey = r_regionkey
WHERE r_name = 'ASIA'
LIMIT 5;

```

```

tpch_db=# SELECT
    c_name,
    c_phone,
    n_name,
    r_name
FROM
    customer
JOIN
    nation ON c_nationkey = n_nationkey
JOIN
    region ON n_regionkey = r_regionkey
WHERE
    r_name = 'ASIA'
LIMIT 5;

```

c_name	c_phone	n_name	r_name
Customer#000000009	18-338-906-3675	INDIA	ASIA
Customer#000000021	18-902-614-8344	INDIA	ASIA
Customer#000000028	18-774-241-1462	INDIA	ASIA
Customer#000000037	18-385-235-7162	INDIA	ASIA
Customer#000000091	18-239-400-3677	INDIA	ASIA

(5 行记录)

```

-- 统计每个地区的供应商数量
-- join 将每个 supplier 与 nation 和 region 关联起来
-- groupby 将所有来自同一 r_name 的供应商记录分组，降序排列结果
SELECT r_name, count(s_suppkey) AS supplier_count
FROM supplier
JOIN nation ON s_nationkey = n_nationkey
JOIN region ON n_regionkey = r_regionkey
GROUP BY r_name
ORDER BY supplier_count DESC;

```

```

tpch_db=# SELECT
    r_name,
    count(s_suppkey) AS supplier_count
FROM
    supplier
JOIN
    nation ON s_nationkey = n_nationkey
JOIN
    region ON n_regionkey = r_regionkey
GROUP BY
    r_name
ORDER BY
    supplier_count DESC;

```

r_name	supplier_count
ASIA	27
AFRICA	21
AMERICA	20
EUROPE	20
MIDDLE EAST	12

(5 行记录)

3. 实验总结

本次实验通过在 macOS 上安装配置 PostgreSQL，掌握了 DBMS 的基本搭建方法。

实验中，我亲手执行了 DDL 语句来构建 TPC-H 数据模型，并使用 `ALTER TABLE` 添加主外键约束，深刻理解了数据完整性在数据库设计中的重要性。

使用 `COPY` 命令批量导入数据的过程，让我体验了真实场景下的数据加载操作。

最后，通过编写并执行从简单到复杂的多表连接 SQL 查询，验证了数据库的正确性，并巩固了 SQL 查询技能。